

UNIDAD 5

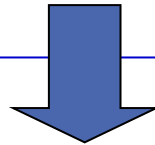


Programación Procedural Listas

LISTAS

Una lista es un **conjunto de elementos del mismo tipo** que puede crecer o contraerse sin restricciones.

Todos los elementos son accesibles y se puede insertar y suprimir un elemento en cualquier posición de la lista.



En **C o Pascal**, es posible trabajar con listas, como una colección de objetos del mismo tipo de datos.

LISTAS

Implementación

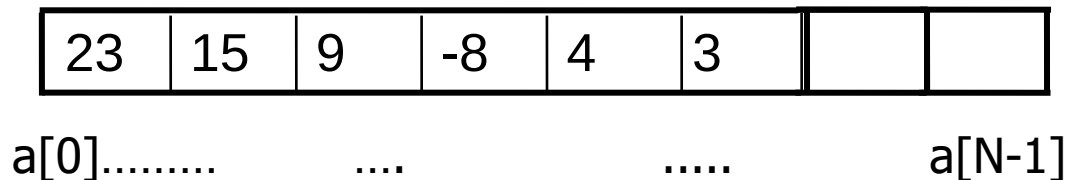
de listas

Mediante arreglos: Listas secuenciales

Mediante Punteros: Listas enlazadas

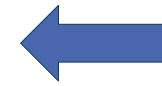
Implementación de listas mediante arreglos: Listas secuenciales

`int * a    o   int a[N];`



Mediante arreglos: Listas secuenciales

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main() {
5      int *arr;
6      int size, i;
7
8      // Pedir tamaño del arreglo al usuario
9      printf("Introduce el tamaño del arreglo: ");
10     scanf("%d", &size);
11
12     // Crear arreglo con tamaño especificado, utilizando malloc
13     arr = (int*) malloc(size * sizeof(int));
14
15     // Si malloc no pudo reservar la memoria, agregaría error
16     if(arr == NULL) {
17         printf("Error de asignacion de memoria.\n");
18         return -1;
19     }
20
21     // Pedir valores para llenar el arreglo
22     printf("Introduce los %d elementos del arreglo:\n", size);
23     for(i = 0; i < size; i++) {
24         scanf("%d", &arr[i]);
25     }
26
27     // Mostrar el contenido del arreglo
28     printf("Los elementos del arreglo son:\n");
29     for(i = 0; i < size; i++) {
30         printf("%d ", arr[i]);
31     }
32
33     // Liberar la memoria reservada para el arreglo
34     free(arr);
35
36     return 0;
37 }
```



Código :
Arreglo Dinámico

Ejecución en Terminal



```
Introduce el tamaño del arreglo: 4
Introduce los 4 elementos del arreglo:
40
30
20
10
Los elementos del arreglo son:
40 30 20 10
```

Almacenamiento Estático y Almacenamiento Dinámico

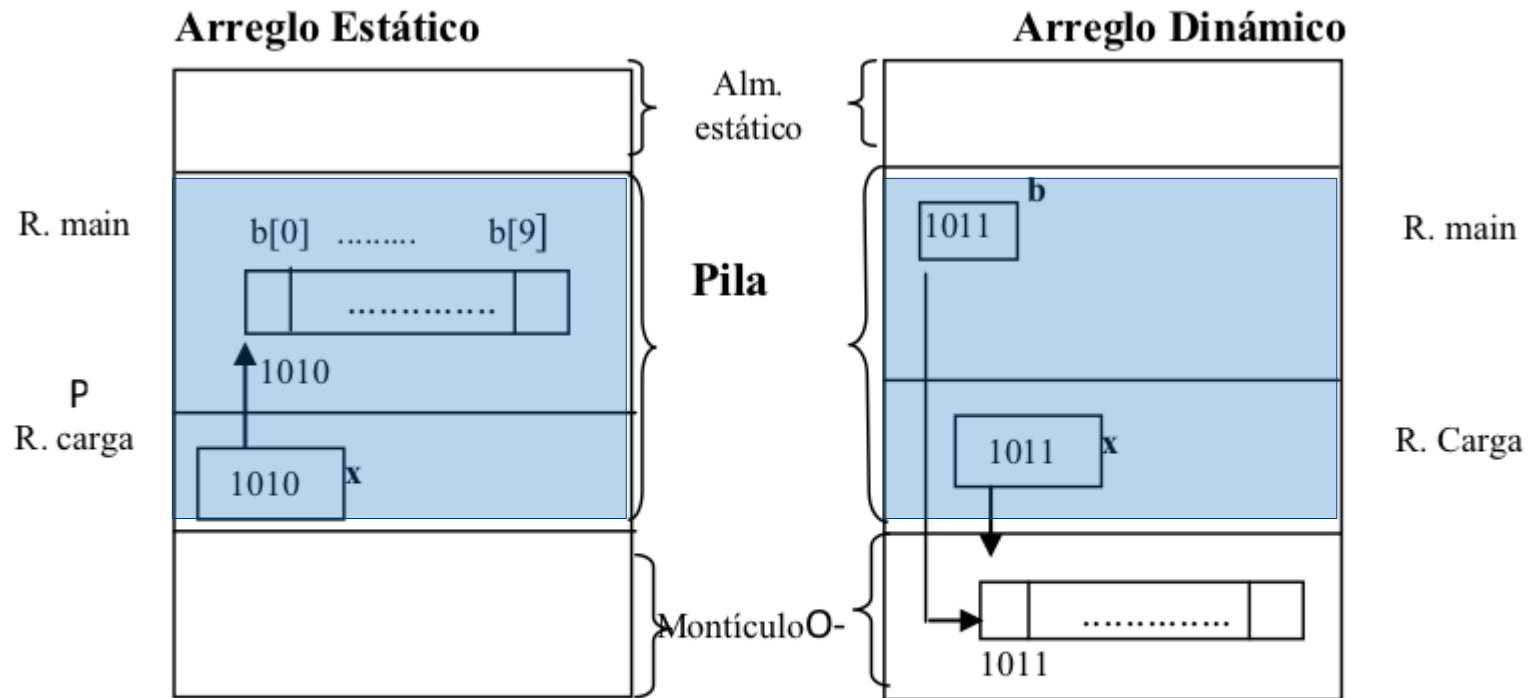


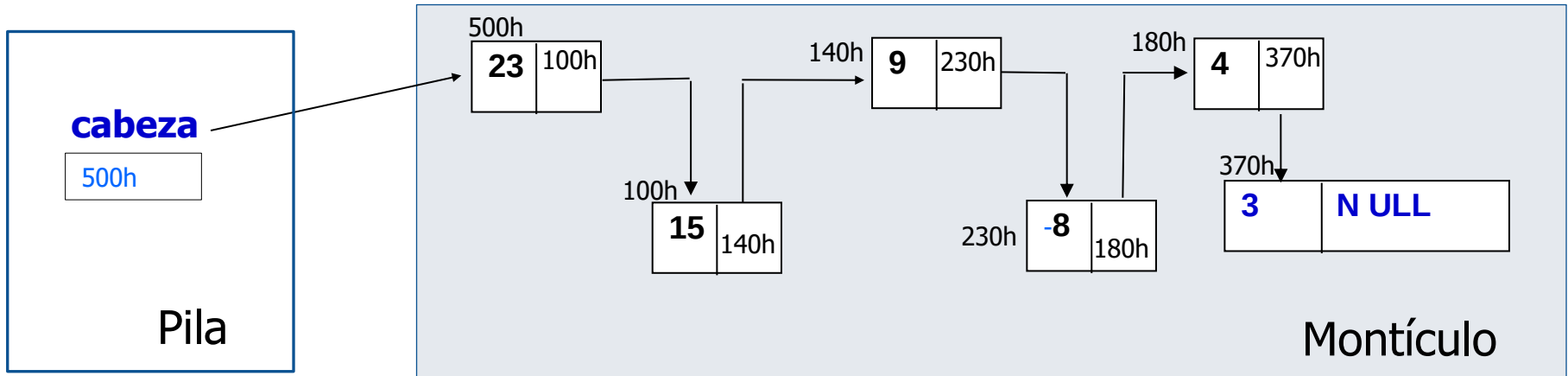
Fig. 2 Organización de la memoria cuando se genera un arreglo estático y un arreglo dinámico

Características de Listas secuenciales

- Se debe conocer cantidad de elementos o asignar suficiente espacio de memoria, si la cantidad no se conoce a priori.
- Los elementos se almacenan en celdas contiguas de memoria.
- Sus elementos se acceden de manera directa
- Insertar un elemento en cualquier lugar de la lista, obliga al desplazamiento de una posición a todos los elementos que le siguen.
- La eliminación de un elemento, excepto el último, requiere desplazamientos para llenar el vacío formado.

Listas implementadas mediante punteros : **Listas enlazadas**

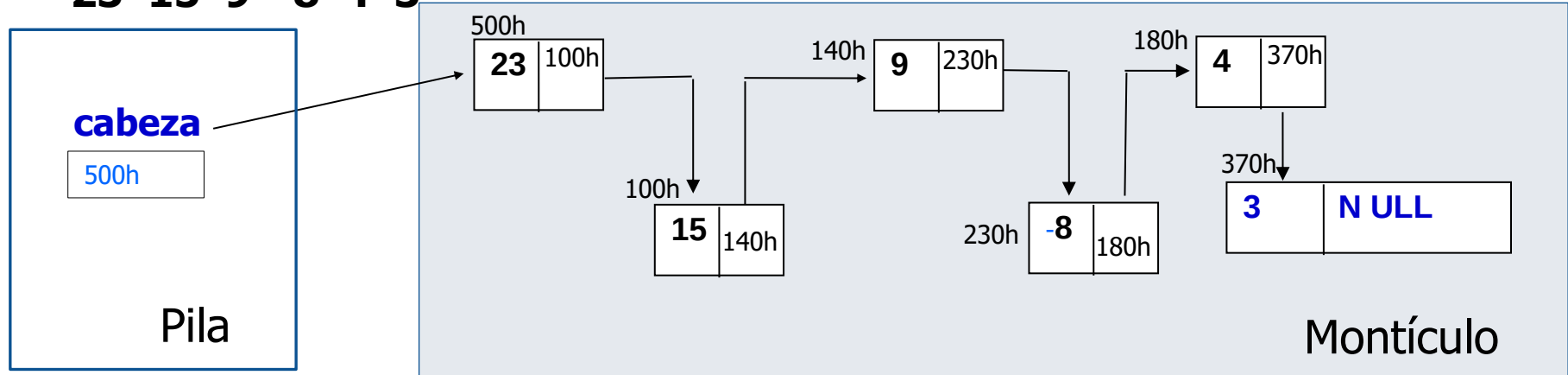
23 15 9 -8 4 3



- Cada elemento deberá contener además de su información intrínseca, un **puntero** a otro elemento de estructura similar.
- El campo enlace apunta al siguiente nodo de la lista excepto el último nodo que contiene NULL, para indicar el final de la lista. Este valor de apuntador NULL no hace referencia a un lugar de memoria, sino que denota que el apuntador apunta a nada.

Listas implementadas mediante punteros : **Listas enlazadas**

23 15 9 -8 4 3



Estructura de un elemento de una lista

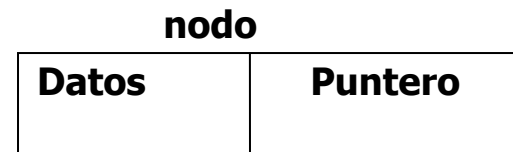
struct nodo

```
{ int nro;
  struct nodo *sig;
};
```

```
typedef struct nodo * puntero;
```

//declaración de variable

```
puntero cabeza;
```



Característica de las Listas enlazadas

En esta representación cada elemento (nodo) de una lista debe tener dos partes:

- ▶ una contiene la información de cualquier tipo de dato simple o estructurado (excepto File)
- ▶ la otra parte, contiene un enlace o puntero que indica la posición del siguiente elemento.

Cada elemento tiene la estructura de tipo *struct*

Los elementos se pueden **almacenar en celdas de memoria no contiguas**

Ventaja:

- ▶ Desaparece el problema de los costosos desplazamientos de elementos para insertar y eliminar componentes de la lista en cualquier posición.
- ▶ No se necesita conocer a priori la cantidad de elementos de la lista.

Desventaja

- ▶ Búsquedas Secuenciales.
- ▶ Requiere un espacio adicional para los apuntadores.

Manipulación de listas enlazadas

Operaciones Básicas

- Creación
- *Inserción*
- *Recorrido*
- *Búsqueda*
- *Modificación*
- *Supresión*
- *Ordenamiento*
- *Eliminación*

Operaciones básicas

Creación

`cabeza = NULL;`

Inserción puede hacerse de tres maneras:

Al comienzo de la lista: inserción por la cabeza de la lista.

Al final de la lista: el nuevo elemento se coloca a continuación del
último elemento de la lista.

Entre otros nodos: el nuevo elemento se inserta en el medio de la lista.

Inserción de un único elemento al comienzo de la lista

```
#include <stdio.h>
#include <stdlib.h> // Para malloc

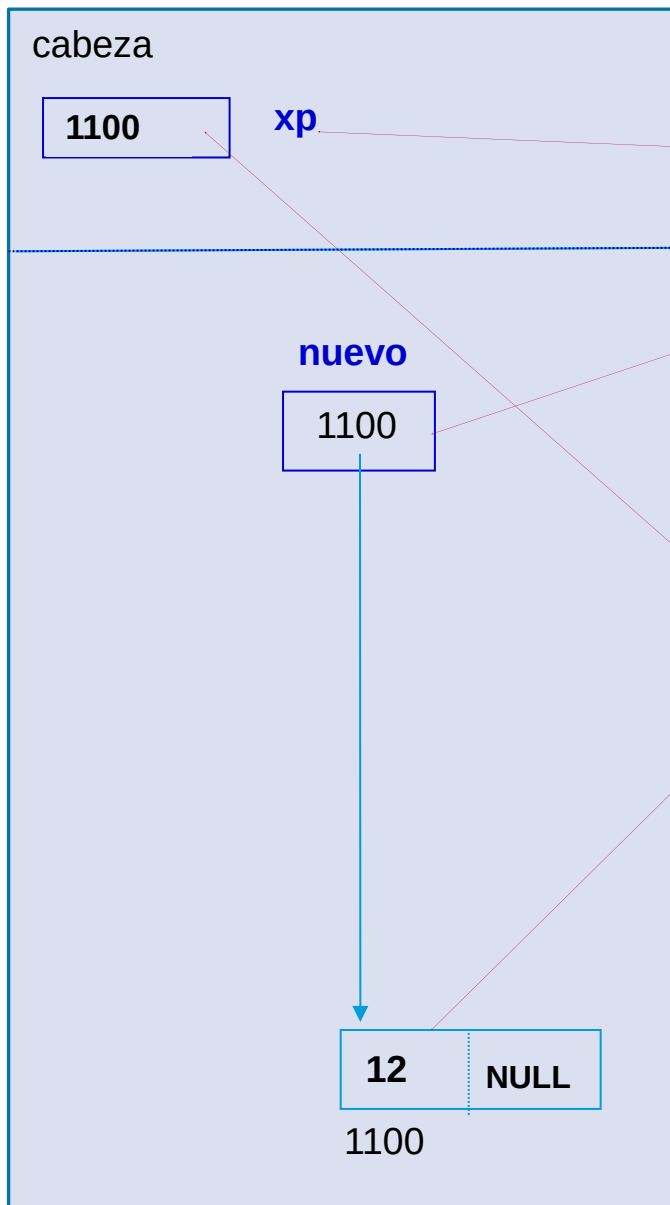
struct nodo {
    int nro;
    struct nodo *sig;
};
typedef struct nodo *puntero;

void insertar(puntero *xp) {
    puntero nuevo;
    nuevo = (puntero) malloc(sizeof(struct nodo));
    printf("\n Ingrese el nuevo valor: ");
    scanf("%d", &nuevo->nro);
    nuevo->sig = *xp;
    *xp = nuevo;
}

void crear(puntero *xp) {
    *xp = NULL;
}

int main(void) {
    puntero cabeza;
    crear(&cabeza);
    insertar(&cabeza);
    printf("valor cabeza de lista agregado: %d ",cabeza->nro);
    return 0;
}
```

Ingrese el nuevo valor: 50
valor cabeza de lista agregado: 50



P
I
L
A

M
O
N
I
T
O
R
E
O

```
void insertar(puntero *xp) {  
    puntero nuevo;  
    nuevo = (puntero) malloc(sizeof(struct nodo));  
    printf("\n Ingrese el nuevo valor: ");  
    scanf("%d", &nuevo->nro);  
    nuevo->sig = *xp;  
    *xp = nuevo;  
}
```

```
int main(void) {  
    puntero cabeza;  
    crear(&cabeza);  
    insertar(&cabeza);  
    printf("valor caberza de lista agregado: %d ",cabeza->nro);  
    return 0;  
}
```



```
Ingrese el nuevo valor: 12  
valor caberza de lista agregado: 12
```

Ejemplo

Lote de Prueba: 8 10 22 54 0

void insertar (puntero &xp)

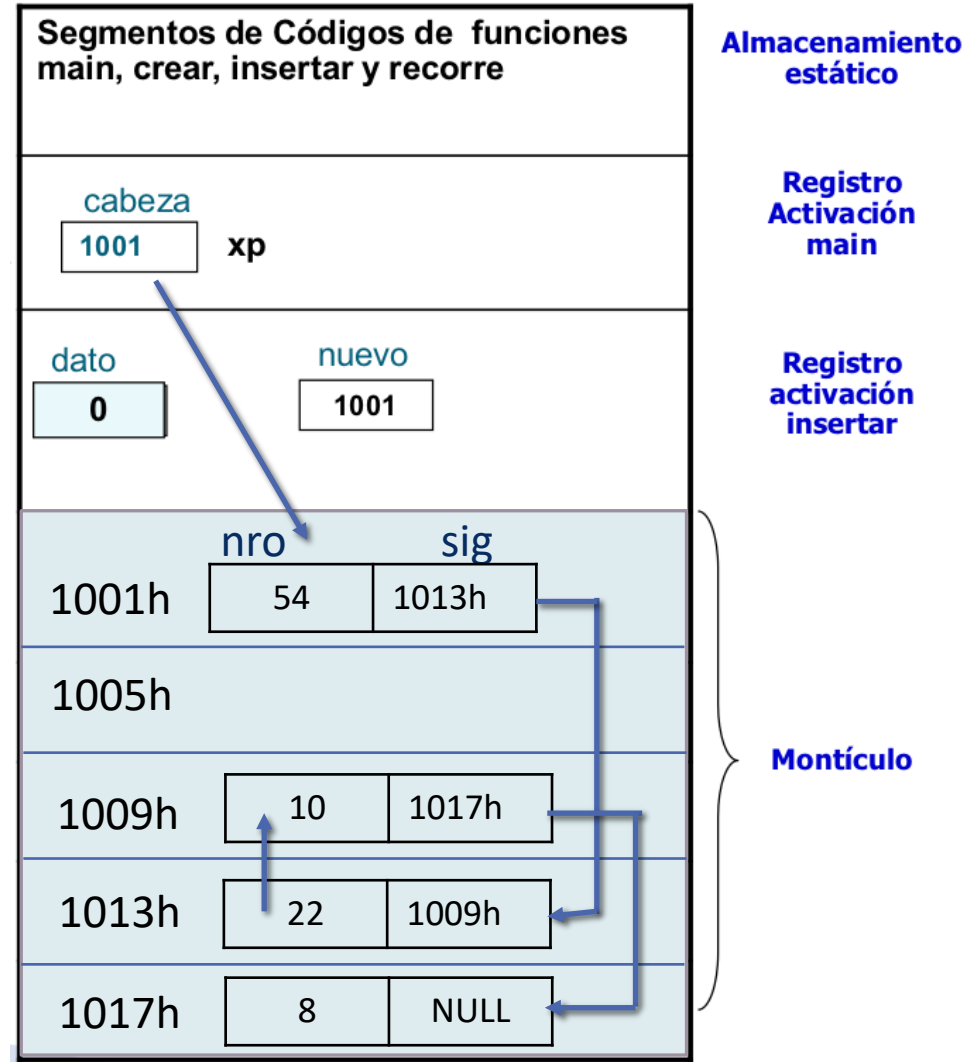
```
{ int dato;  
  puntero nuevo;  
  printf(" Ingrese nuevo valor (0 para  
    terminar)");  
  scanf("%d",&dato);  
  while (dato != 0)  
  {  
    nuevo =(puntero) malloc(sizeof(struct nodo));  
    nuevo->nro = dato;  
    nuevo->sig = xp;  
    xp = nuevo;  
    scanf("%d",&dato);  
  }  
}
```

void crear (puntero &xp)

```
{  
  xp=NULL;  
}
```

void main (void)

```
{  
  puntero cabeza;  
  crear (cabeza);  
  insertar (cabeza);  
}
```



Recorrido de la lista

```
#include <stdio.h>
#include <stdlib.h>

struct nodo {
    int nro;
    struct nodo *sig;
};
typedef struct nodo *puntero;

void insertar(puntero *xp) {
    int dato;
    puntero nuevo;
    printf("Ingrese nuevo valor (0 para terminar): ");
    scanf("%d", &dato);

    while (dato != 0) {
        nuevo = (puntero) malloc(sizeof(struct nodo));
        nuevo->nro = dato;
        nuevo->sig = *xp;
        *xp = nuevo;
        printf("Ingrese nuevo valor (0 para terminar): ");
        scanf("%d", &dato);
    }
}

void recorre (puntero xp)
{
    printf("\n Listado de datos ");
    while (xp != NULL)
    {
        printf("\n %d ",xp->nro);
        xp = xp->sig;
    }
}

void crear(puntero *xp) {
    *xp = NULL;
}

int main(void) {
    puntero cabeza;
    crear(&cabeza);
    insertar(&cabeza);
    recorre(cabeza);
    return 0;
}
```

```
void recorre (puntero xp)
{
    printf("\n Listado de datos ");
    while (xp != NULL)
    {
        printf("\n %d ",xp->nro);
        xp = xp->sig;
    }
}
```

Ejecución en Terminal

```
Ingrese nuevo valor (0 para terminar): 10
Ingrese nuevo valor (0 para terminar): 20
Ingrese nuevo valor (0 para terminar): 30
Ingrese nuevo valor (0 para terminar): 40
Ingrese nuevo valor (0 para terminar): 0
```

```
Listado de datos
40
30
20
10
```

Búsqueda y Modificación en Listas

Agregar al programa anterior:

- ▶ una función que permita **buscar** en la lista un elemento que se ingresa por teclado
 - Un grupo devuelve un resultado al main (1 si está , -1 si no está)
 - Otro grupo escribe mensaje en subprograma
- ▶ una función que permita **modificar** un elemento de la lista

Realice una versión iterativa y una versión recursiva de cada función

En ambos casos indique el tipo de pasaje de parámetros utilizado, justificando la respuesta


```
struct nodo
{
    int nro;
    struct nodo *sig;
};
typedef struct nodo * puntero;
int main ()
{
    puntero cabeza;
    int num, viejo, nuevo;
    cabeza=NULL;
    insertar (cabeza);
    recorre (cabeza);
    printf(" Ingrese el elemento a buscar ");
    scanf("%d",&num);
    busca (cabeza, num);
    buscarecursivo (cabeza, num);
    printf(" Ingrese el valor a modificar y el nuevo valor ");
    scanf("%d",&viejo);
    scanf("%d",&nuevo);
    Modifica (cabeza, viejo, nuevo);
    Modificarecursivo (cabeza, viejo, nuevo);
}
```

void busca (puntero xp, int num)

```
{ while( (xp != NULL )&& (xp->nro != num))
    xp = xp->sig;
  if (xp == NULL)
    printf(" El número no está en la lista");
  else printf(" Se encontró el número en la lista");
}
```

← ITERATIVO

¿POR QUÉ
puntero xp no
pasa con * o
con & ?

void buscarecursivo (puntero xp,int num)

```
{
  if(xp==NULL)
    printf ("el elemento no se encuentra en la lista");
  else if (xp->nro == num)
    printf ("Se encontró el número en la lista");
  else buscarecursivo (xp->sig, num);
}
```

← RECURSIVO

```

void modifica (puntero xp, int v, int n) //v es el viejo número y n es el nuevo
{
    while ((xp != NULL) && (xp->nro != v))
        xp = xp->sig;
    if (xp != NULL)
        xp->nro = n;
    else
        printf("\n el número no está en la lista");
    return;
}

```

← ITERATIVO

Ambos casos
modifican un DATO
EXISTENTE, NO
UN PUNTERO

```

void modifica (puntero xp, int v, int n)
{ if (xp == NULL)
    printf ("\n el numero no está en la lista");
    else if (xp->nro == v)
        { xp->nro = n;
          return;
        }
    else modifica (xp->sig, v, n);
}

```

← RECURSIVO

Las funciones de búsqueda y modificación responden al mismo algoritmo señalado para recorrer la lista:

Inicio Bucle

*Se procesa el nodo
apuntado (si es el
nodo buscado se
informa o se modifica
y se sale del bucle)*

*Se avanza al siguiente
nodo (si no se
encontró el nodo
buscado)*

Fin del Bucle

Otras operaciones sobre listas

- 1 - Inserción de elementos en cualquier lugar de una lista
- 2 - Supresión de un elemento en una lista / Eliminación de todas las componentes de una lista
- 3 - Ordenamiento de lista (ver libro de cátedra)

Otras funciones recursivas en listas

Ejecución en consola de comandos



```
#include <stdio.h>
#include <stdlib.h>

struct nodo {
    int nro;
    struct nodo *sig;
};

typedef struct nodo *puntero;
void insertar(puntero *xp) {
    int dato;
    puntero nuevo;
    printf("Ingrese un numero entero (0 para terminar):\n");
    scanf("%d", &dato);
    if (dato) {
        nuevo = (puntero)malloc(sizeof(struct nodo));

        // Comprobar si la asignación de memoria tuvo éxito
        if (nuevo == NULL) {
            printf("Error al asignar memoria.\n");
            return;
        }
        nuevo->nro = dato;
        nuevo->sig = *xp;
        *xp = nuevo;
        insertar(xp);
    } else {
        return;
    }
}

void listar(puntero xp)
{
    if (xp != NULL) {
        printf("\n%d ", xp->nro);
        xp = xp->sig;
        listar(xp);
    } else {
        return;
    }
}

int main(void) {
    puntero cabeza = NULL; // inicialización a NULL
    insertar(&cabeza);
    listar(cabeza);
    return 0;
}
```

```
Ingrese un numero entero (0 para terminar):
10
Ingrese un numero entero (0 para terminar):
20
Ingrese un numero entero (0 para terminar):
30
Ingrese un numero entero (0 para terminar):
40
Ingrese un numero entero (0 para terminar):
0

40
30
20
10
```

Listas enlazadas que almacenan datos estructurados

Generar una lista con los datos de los empleados de una empresa: legajo y sueldo. El ingreso de información termina con legajo cero(0). Mostrar el legajo de los empleados almacenados en la lista, cuyo sueldo es superior a 1000.

```
#include <stdio.h>
#include <alloc.h>
```

```
typedef struct
```

```
{  int legajo;
    float sueldo;
} empleado ;
```

```
struct nodo
```

```
{
    empleado dato;
    struct nodo * sig;
};
```

```
typedef struct nodo *puntero;
```

```
void insertar (puntero *xco)
```

```
{ puntero nuevo;
  int num;
  printf("\n Ingrese legajo : ");
  scanf ("%d", &num);
  if( num )
  { nuevo = (puntero) malloc(sizeof(struct nodo));
    nuevo->dato.legajo = num;
    printf("\n Ingrese sueldo : ");
    scanf ("%f", &nuevo->dato.sueldo);
    nuevo->sig = xco;
    xco = nuevo;
    insertar (xco)
  }
  return;
}
```

Arreglo de Listas

La Facultad de Ciencias Exactas organizó el Congreso de Informática y necesita contar con un programa que le permita administrar la información relativa a los 10 tutoriales que se proponen en dicho evento. Para ello, el programa deberá permitir, a través de un menú de opciones, los siguientes ítems:

- Ingresar los datos correspondientes a cada tutorial, a saber: número de tutorial (1-10), título.
- Registrar las inscripciones, ingresando el DNI del inscripto y el número de tutorial al que desea asistir.
- Eliminar alguna inscripción, en cuyo caso se ingresarán los mismos datos que en el ítem anterior.
- Dado el número o el título de un tutorial, mostrar los datos específicos del mismo, incluido la cantidad inscriptos.
- Dado el DNI de una persona, informar el/los tutoriales (número y título) en los que se inscribió.



struct nodo

```
{  
    int dni;  
    struct nodo * sig;  
};
```

```
typedef char cadena[30];
```

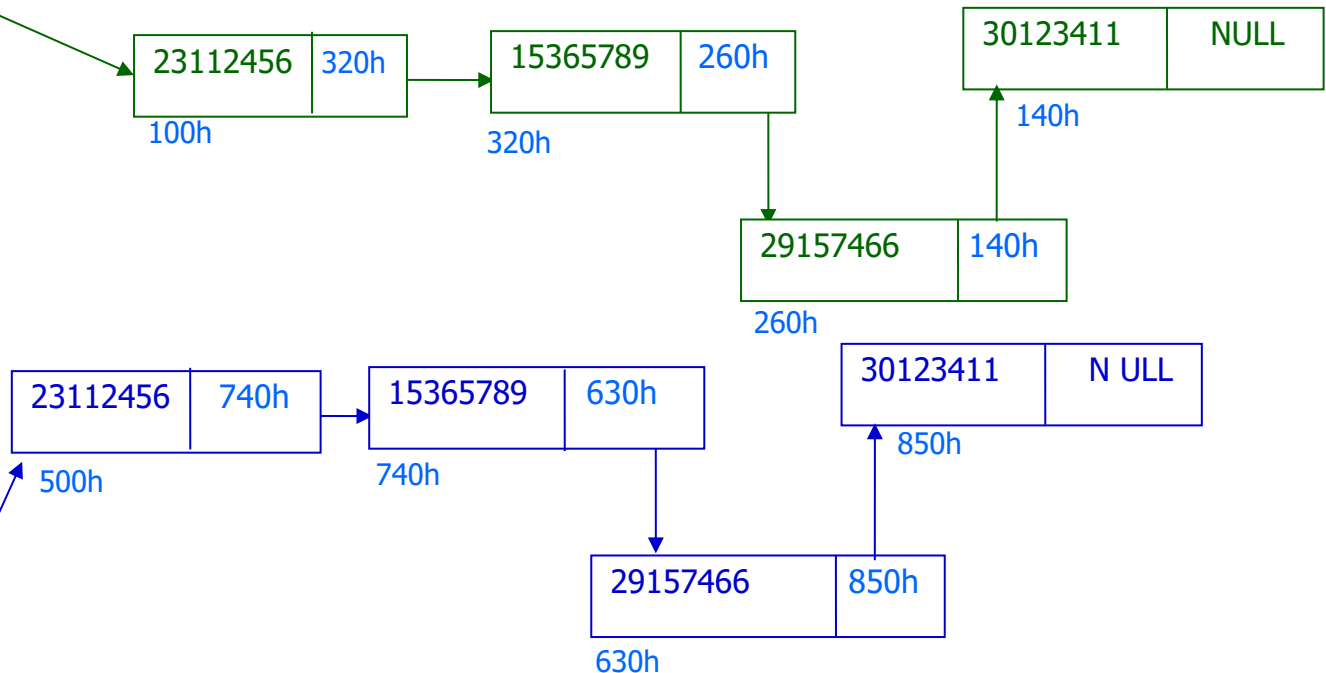
struct datos

```
{ int numero;  
  cadena titulo;  
  struct nodo * ins;  
};
```

```
typedef struct nodo * inscriptos;
```

datos tutorial[10];

1	Diseño OO	100h
2	C#	NULL
...		
10	Prolog	500h



```

#include<stdio.h>
#include<malloc.h>

const int fila=3;

struct nodo
{
    int dni;
    struct nodo * sig;
};

typedef char cadena[30];

struct datos
{ int numero;
  cadena titulo;
  struct nodo * ins;
};

typedef struct nodo * inscriptos;

void crear (datos &d)
{
    d.ins=NULL;
}

```

```

void insertar (inscriptos *xp, int DNI)

```

```

{
    inscriptos nuevo;
    nuevo = (inscriptos) malloc(sizeof(struct nodo));
    nuevo->dni=DNI;
    nuevo->sig = xp;
    xp = nuevo;
    return;
}

```

```

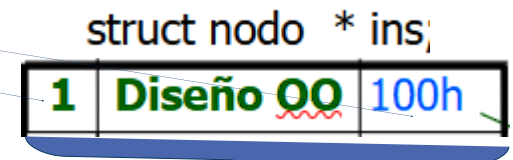
void listar(datos t[])

```

```

{ int i;
  inscriptos xp;
  for(i=0; i< fila; i++)
  {
      printf("\n Datos de Inscriptos en el Tutorial:%3d\n", i+1);
      printf("\n TITULO: "); puts(t[i].titulo);
      printf("\n INSCRIPTOS \n ");
      xp= t[i].ins;
      while (xp!= NULL)
      {
          printf("\n %d",xp->dni);
          xp = xp->sig;
      }
  }
}

```



componente "i"
del arreglo

```

int main(void)
{ int i, n, op;
  long d;
  datos tutorial[filas]; /* arreglo de tutoriales */
  printf("\n ***** Cargar de los tutoriales C \n *****");
  for (i=0;i<filas;i++)
  {
    tutorial[i].numero = i+1;
    printf ("\n Ingrese nombre del tutorial %d :",i+1);
    gets (tutorial[i].titulo);
    crear (tutorial[i]);
  }
  do { system ("cls");
    printf ("\n ***** MENU DE OPCIONES *****\n");
    printf ("\n 1 - Realizar una inscripción: ");
    printf ("\n 2 - Eliminar una inscripción: ");
    printf ("\n 3 - Mostrar Datos de un tutorial: ");
    printf ("\n 4 - Mostrar tutoriales en los cuales se inscribió una persona : ");
    printf ("\n 5 - Mostrar tutoriales con Datos de Inscriptos : ");
    printf ("\n 6 - Salir del Menú: ");
    scanf ("%d", &op);
    switch (op)
    { case 1: { printf("\n Ingrese numero del tutorial :");
                scanf("%d", &n);
                printf("\n Ingrese DNI :");
                scanf("%d", &d);
                insertar (tutorial[n-1].ins, nd);
                break;
            } // complete los rotulos faltantes.....
      case 5: listar(tutorial) ;
    }
  } while (op !=6);
}

```